

Chapter 5: Advanced Control Flow

By Irina Galata

In the previous chapter, you learned how to control the flow of execution using the decision-making powers of `if` expressions and the `while` loop. In this chapter, you'll continue to learn how to control the flow of execution. You'll learn about another loop known as the `for` loop.

Loops may not sound very interesting, but they're very common in computer programs. For example, you might have code to download an image from the cloud; with a loop, you could run that multiple times to download your entire photo library. Or if you have a game with multiple computer-controlled characters, you might need a loop to go through each one and make sure it knows what to do next.

You'll also learn about `when` expressions, which are particularly powerful in Kotlin. They let you inspect a value and decide what to do based on that value. They're incredibly powerful when used with argument matching.

Open of the starter project for this chapter or create a fresh project to get going.

Ranges

Before you dive into the `for` loop statement, you need to know about the **range** data types, which let you represent a sequence of countable integers. Let's look at two types of ranges.

First, there's a **closed range**, which you represent like so:

```
val closedRange = 0..5
```

The two dots (`..`) indicate that this range is closed, which means the range goes from 0 to 5 inclusive. That's the numbers (0, 1, 2, 3, 4, 5).

Second, there's a **half-open range**, which you represent like so:

```
val halfOpenRange = 0 until 5
```

Here, you replace the two dots with `until`. Half-open means the range goes from 0 up to, but not including, 5. That's the numbers (0, 1, 2, 3, 4).

Open and half-open ranges created with the `..` and `until` operators are always increasing. In other words, the second number must always be greater than or equal to the first. To create a decreasing range, you can use `downTo`, which is inclusive:

```
val decreasingRange = 5 downTo 0
```

That will include the numbers (5, 4, 3, 2, 1, 0).

Ranges are commonly used in both `for` loops and when expressions, which means that throughout the rest of the chapter, you'll use ranges as well!

For loops

In the previous chapter you looked at `while` loops. Now that you know about ranges, it's time to look at another type of loop: The **for loop**. This is probably the most common loop you'll see, and you'll use it to run code a certain number of times.

You construct a `for` loop like this:

```
for (/* VARIABLE */ in /* RANGE */) {  
    /* LOOP CODE */  
}
```

The loop begins with the `for` keyword, followed by a name given to the loop variable (more on that shortly), followed by `in`, followed by the range to loop through. Here's an example:

```
val count = 10

var sum = 0
for (i in 1..count) {
    sum += i
}
```

In the code above, the `for` loop iterates through the range 1 to `count`. At the first iteration, `i` will equal the first element in the range: 1. Each time around the loop, `i` will increment until it's equal to `count`; the loop will execute one final time and then finish.

Note: If you'd used a half-open range, the last iteration would see `i` equal to `count - 1`.

Inside the loop, you add `i` to the `sum` variable; it runs 10 times to calculate the sequence `1 + 2 + 3 + 4 + 5 + ...` all the way up to 10.

Here are the values of the constant `i` and variable `sum` for each iteration:

- **Start of iteration 1:** `i = 1`, `sum = 0`
- **Start of iteration 2:** `i = 2`, `sum = 1`
- **Start of iteration 3:** `i = 3`, `sum = 3`
- **Start of iteration 4:** `i = 4`, `sum = 6`
- **Start of iteration 5:** `i = 5`, `sum = 10`
- **Start of iteration 6:** `i = 6`, `sum = 15`
- **Start of iteration 7:** `i = 7`, `sum = 21`
- **Start of iteration 8:** `i = 8`, `sum = 28`
- **Start of iteration 9:** `i = 9`, `sum = 36`
- **Start of iteration 10:** `i = 10`, `sum = 45`
- **After iteration 10:** `sum = 55`

In terms of scope, the `i` constant is only visible inside the scope of the `for` loop, which means it's not available outside of the loop.

Repeat loops

Finally, sometimes you only want to loop a certain number of times, and so you don't need to use the loop constant at all. In that case, you can employ a repeat loop, like so:

```
sum = 1
var lastSum = 0
repeat(10) {
    val temp = sum
    sum += lastSum
    lastSum = temp
}
```

Steps in loops

It's also possible to only perform certain iterations in the range. For example, imagine you wanted to compute a sum similar to that of triangle numbers, but only for odd numbers:

```
sum = 0
for (i in 1..count step 2) {
    sum += i
}
```

The previous loop has a step operator in the `for` loop statement. The loop will only run through the values that the step falls on. In this case, rather than step through every value in the range, it will step through every other value. As such, `i` will always be odd because the starting value is 1.

You can even count down in a `for` loop using `downTo`. In this case if count is 10 then the loop will iterate through the following values (10, 8, 6, 4, 2).

```
sum = 0
for (i in count downTo 1 step 2) {
    sum += i
}
```

Labeled statements

Sometimes you'd like to skip a loop iteration for a particular case without breaking out of the loop entirely. You can do this with the `continue` statement, which immediately ends the current iteration of the loop and starts the next iteration. The `continue` statement gives you a higher level of control, letting you decide where and when you want to skip an iteration.

Take the example of an 8 by 8 grid, where each cell holds a value of the row multiplied by the column. It looks much like a multiplication table, doesn't it?

	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	8	10	12	14
3	0	3	6	9	12	15	18	21
4	0	4	8	12	16	20	24	28
5	0	5	10	15	20	25	30	35
6	0	6	12	18	24	30	36	42
7	0	7	14	21	28	35	42	49

Let's say you wanted to calculate the sum of all cells but exclude all even rows, as shown below:

	0	1	2	3	4	5	6	7
0								
1	0	1	2	3	4	5	6	7
2								
3	0	3	6	9	12	15	18	21
4								
5	0	5	10	15	20	25	30	35
6								
7	0	7	14	21	28	35	42	49

Using a `for` loop, you can achieve this as follows:

```
sum = 0
for (row in 0 until 8) {
    if (row % 2 == 0) {
        continue
    }
}
```

```

    for (column in 0 until 8) {
        sum += row * column
    }
}

```

When the row modulo 2 equals 0, the row is even. In this case, `continue` makes the for loop skip to the next row.

The `break` statement you saw in the last chapter used with while loops also works with for loops, and takes you to the next statement after the for loop. Just like `break`, `continue` works with both for loops and while loops.

The second code example will calculate the sum of all cells, excluding those where the column is greater than or equal to the row.

To illustrate, it should sum the following cells:

	0	1	2	3	4	5	6	7
0								
1	0							
2	0	2						
3	0	3	6					
4	0	4	8	12				
5	0	5	10	15	20			
6	0	6	12	18	24	30		
7	0	7	14	21	28	35	42	

Using a for loop, you can achieve this as follows:

```

sum = 0
rowLoop@ for (row in 0 until 8) {
    columnLoop@ for (column in 0 until 8) {
        if (row == column) {
            continue@rowLoop
        }
        sum += row * column
    }
}

```

The previous code block makes use of a **label**, labeling the two loops as `rowLoop` and `columnLoop`, respectively. When the row equals the column inside the inner `columnLoop`, the outer `rowLoop` will continue.

You can use labeled statements like these with `break` to break out of a certain loop. Normally, `break` and `continue` work on the innermost loop, so you need to use labeled statements if you want to manipulate an outer loop.

Mini-exercises

1. Create a constant named `range` and set it equal to a range starting at 1 and ending with 10 inclusive. Write a `for` loop which iterates over this range and prints the square of each number.
2. Write a `for` loop to iterate over the same range as in the exercise above and print the square root of each number. You'll need to type convert your loop constant.
3. Above, you saw a `for` loop which iterated over only the even rows like so:

```
sum = 0
for (row in 0 until 8) {
    if (row % 2 == 0) {
        continue
    }

    for (column in 0 until 8) {
        sum += row * column
    }
}
```

Change this to use a step clause on the first `for` loop to skip even rows instead of using `continue`. Check that the sum is 448 as in the initial example.

When expressions

You can also control flow via the `when` expression. It executes different code depending on the value of a variable or constant. Here's a `when` expression that acts on an integer:

```
val number = 10

when (number) {
    0 -> println("Zero")
    else -> println("Non-zero")
}
```

In this example, the code will print the following:

```
Non-zero
```

The purpose of this when expression is to determine whether or not a number is zero. It will get more complex — I promise!

To handle a specific case, you add the value followed by `->` which indicates the code that will execute if the condition is met. Then, you use `else` to signify what should happen for all other values. Unlike other languages such as Java, there is no need to include a `break` statement in each branch, as the `when` expression will only use the first matching branch and then return.

Here's another example:

```
when (number) {  
    10 -> println("It's ten!")  
}
```

This time you check for 10, in which case, you print a message. Nothing should happen for other values.

Of course, when expressions also work with data types other than integers. Here's an example using a string:

```
val string = "Dog"  
when (string) {  
    "Cat", "Dog" -> println("Animal is a house pet.")  
    else -> println("Animal is not a house pet.")  
}
```

This will print the following:

```
Animal is a house pet.
```

In this example, you provide two values for the first argument, meaning that if the value is equal to either "Cat" or "Dog", then the statement will execute that branch of the expression.

Returning values

You can also give your `when` expressions more than one branch. And, due to the fact that `when` is an expression, you can use it to return a value, just like you can with `if` expressions. You can also ignore the value if you want to just use `when` as a statement. A `when` expression will return the value from the first branch with a matching argument.

If you want to determine the name of the number, you can assign the value with a `when` expression as follows

```
val numberName = when (number) {  
    2 -> "two"  
    4 -> "four"  
    6 -> "six"  
    8 -> "eight"  
    10 -> "ten"  
    else -> {  
        println("Unknown number")  
        "Unknown"  
    }  
}  
  
println(numberName) // > ten
```

In the `else` branch, you've used braces to include an entire block of code in the branch. The last value in the block is returned from the branch, so if `number` were not in (2, 4, 6, 8, 10) then `numberName` would have the value "Unknown".

Advanced when expressions

In the previous chapter, you saw an `if` expression that used multiple `else` clauses to convert an hour of the day to a string describing that part of the day. You could rewrite that with a `when` expression, like so:

```
val hourOfDay = 12  
val timeOfDay: String  
  
timeOfDay = when (hourOfDay) {  
    0, 1, 2, 3, 4, 5 -> "Early morning"  
    6, 7, 8, 9, 10, 11 -> "Morning"  
    12, 13, 14, 15, 16 -> "Afternoon"  
    17, 18, 19 -> "Evening"  
    20, 21, 22, 23 -> "Late evening"  
    else -> "INVALID HOUR!"  
}  
println(timeOfDay)
```

This code will print the following:

```
Afternoon
```

Using when expressions with ranges

Remember ranges? Well, you can use ranges to simplify this when expression. You can rewrite the above code using ranges:

```
timeOfDay = when (hourOfDay) {  
    in 0..5 -> "Early morning"  
    in 6..11 -> "Morning"  
    in 12..16 -> "Afternoon"  
    in 17..19 -> "Evening"  
    in 20..23 -> "Late evening"  
    else -> "INVALID HOUR!"  
}
```

This is more succinct than writing out each value individually for all branches.

When there are multiple branches, the expression will execute the first one that matches. You'll probably agree that this is more succinct and more clear than using an if expression for this example.

It's slightly more precise as well, because the if expression method didn't address negative numbers, which here are correctly deemed to be invalid.

Using when expressions with conditions

It's also possible to match a branch to a condition based on a property of the value, without any argument being supplied to the when expression. As you learned in Chapter 2, you can use the modulo operator to determine if an integer is even or odd.

Consider this code:

```
when {  
    number % 2 == 0 -> println("Even")  
    else -> println("Odd")  
}
```

This will print the following:

```
Even
```

The first branch of this when expression uses the `==` operator, meaning the argument will match only when a certain condition is true.

In this example, you’ve designed the argument to match if the value is even — that is, if the value modulo 2 equals 0.

Another example of using conditions in when expressions to great effect is as follows:

```
val (x, y, z) = Triple(3, 2, 5)

when {
    x == 0 && y == 0 && z == 0 -> println("Origin")
    y == 0 && z == 0 -> println("On the x-axis at x = $x")
    x == 0 && z == 0 -> println("On the y-axis at y = $y")
    x == 0 && y == 0 -> println("On the z-axis at z = $z")
    else -> println("In space at x = $x, y = $y, z = $z")
}
```

Here’s what each branch does, in order:

1. Matches precisely the case where the value is $(0, 0, 0)$. This is the origin of 3D space.
2. Matches $y=0, z=0$ and any value of x . This means the coordinate is on the x -axis.
3. Matches $x=0, z=0$ and any value of y . This means the coordinate is on the y -axis.
4. Matches $x=0, y=0$ and any value of z . This means the coordinate is on the z -axis.
5. Matches the remainder of coordinates.

The final branch with the `else` clause is the default; it matches anything, because there are no constraints on any part of the coordinates. Here’s an example of a more complex case:

```
when {
    x == y -> println("Along the y = x line.")
    y == x * x -> println("Along the y = x^2 line.")
}
```

Here, you match the “ y equals x ” and “ y equals x squared” lines. Since there is no argument to the when expression, you do not need an `else` branch, and the when expression will not execute a branch if there is no matching condition. And those are the basics of when expressions!

Mini-exercises

1. Write a when expression that takes an age as an integer and prints out the life stage related to that age. You can make up the life stages, or use the categorization as follows: 0-2 years, Infant; 3-12 years, Child; 13-19 years, Teenager; 20-39, Adult; 40-60, Middle aged; 61+, Elderly.
2. Write a when expression that destructures a Pair containing a string and an integer. The string is a name, and the integer is an age. Use the same cases that you used in the previous exercise to print out the name followed by the life stage. For example, for myself it would print out "Joe is an adult."

Challenges

1. In the following for loop, what will be the value of sum, and how many iterations will happen?

```
var sum = 0
for(i in 0..5) {
    sum += i
}
```

2. In the while loop below, how many instances of "a" will there be in aLotOfAs? Hint: aLotOfAs.count tells you how many characters are in the string aLotOfAs.

```
var aLotOfAs = ""
while (aLotOfAs.length < 10) {
    aLotOfAs += "a"
}
```

3. Consider the following when expression:

```
when {
    x == y && y == z -> println("x = y = z")
    z == 0 -> println("On the x/y plane")
    y == 0 -> println("On the x/z plane")
    x == 0 -> println("On the y/z plane")
    else -> println("Nothing special")
}
```

What will this code print when coordinates is each of the following?

```
val (x, y, z) = Triple(1, 5, 0)
val (x, y, z) = Triple(2, 2, 2)
val (x, y, z) = Triple(3, 0, 1)
val (x, y, z) = Triple(3, 2, 5)
val (x, y, z) = Triple(0, 2, 4)
```

4. A closed range can never be empty. Why?
5. Print a countdown from 10 to 0 without using `downTo`.
6. Print 0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0.

Note: You may find some rounding issues with Challenge #6 where the decimals are slightly off. For example, 0.3 might print out as something like 0.30000000000000004. This is okay for this solution. The reason for this is because of the way computers handle numbers and is too complex for this chapter.

Key points

- You can use **ranges** to create a sequence of numbers, incrementing to move from one value to another.
- **Closed ranges** include both the start and end values.
- **Half-open ranges** include the start value and stop one before the end value.
- **For loops** allow you to iterate over a range.
- The **continue** statement lets you finish the current iteration of a loop and begin the next iteration.
- **Labeled statements** let you use `break` and `continue` on an outer loop.
- You use **when** expressions to decide which code to run depending on the value of a variable or constant.
- The power of a `when` expression comes from leveraging **pattern matching** to compare values using complex rules.

Where to go from here?

You've learned about the core language features for dealing with data over these past few chapters, from data types to variables, then on to decision-making with Booleans and loops with ranges. In the next chapter you'll learn one of the key ways to make your code more reusable and easy to read through the use of functions.